Comparative Performance Evaluation of Lexical, Neural and Hybrid Ranking for E-Commerce Product Search on the Amazon ESCI Dataset

Students Name	Registration No.	Email-id
Rasagya Vatsal	23FE10CSE00716	Rasagya.23fe10cse00716@muj.manipal.edu
Keshav Agrawal	23FE10CSE00810	Keshav.23fe10cse00810@muj.manipal.edu
Shivansh Dhyani	23FE10CSE00722	Shivansh.23fe10cse00722@muj.manipal.edu

Abstract — Product search is harder than it looks. Shopping queries are short, packed with brand names, model numbers and sizes, and a product can be useful to the shopper without being exactly what was typed. This report compares lexical, neural and hybrid ranking on the Amazon Shopping Queries (ESCI) dataset, in which every judged query-product pair is labelled Exact, Substitute, Complement or Irrelevant. We index the 482,105 US products that carry a judgment and evaluate TF-IDF, Okapi BM25, a Sentence Transformer (all-MiniLM-L6-v2) and three Reciprocal Rank Fusion (RRF) hybrids on all 8,956 test queries, retrieving from the full catalog. BM25 is the strongest and fastest single ranker (nDCG@10 = 0.320). Standard RRF of BM25 and the dense model lifts Recall@100 from 0.567 to 0.604 but leaves the top ten unchanged. A weighted RRF, whose BM25 weight and rank constant were tuned on 2,000 training queries, is the best method on precision-oriented metrics: nDCG@10 = 0.331, MAP = 0.261 and P@10 = 0.216, roughly 4% above BM25, a gain that is statistically significant (paired t-test $p < 10^{-50}$). Adding TF-IDF as a third fused ranker lowers every metric.

Keywords: *Information Retrieval · E-Commerce Product Search · Amazon ESCI · BM25 · Sentence Transformers · Reciprocal Rank Fusion*

1. Introduction

Search is the main way customers find things in an online store. A shopper types a few words and expects the right products on the first screen; anything ranked below that is rarely seen. Product search differs from classic document retrieval in three ways. Queries are short and specific ("hp 201a black", "tech deck halfpipe"). The "documents" are product titles of about twenty words rather than paragraphs of prose. And relevance is not binary: a different model of the same headset may be an acceptable substitute, and a case for the right phone is a complement rather than a match.

Lexical rankers such as TF-IDF and Okapi BM25 [1, 2] score products by the query terms they share, weighted by how rare each term is. They are fast, need no training and are very good at the tokens that matter most in shopping: brands, model numbers and sizes. Their weakness is vocabulary mismatch. A misspelled query, a paraphrase, or a word removed during preprocessing leaves them with nothing to match. Dense retrievers based on Sentence Transformers [3] encode the query and each product into a shared vector space, so they can match on meaning rather than surface form and cope with typos. A general-purpose model, however, tends to blur exactly the fine-grained distinctions ("201a" versus "202a") that a shopper cares about.

Because the two families fail on different queries, combining them is attractive. Reciprocal Rank Fusion (RRF) [4] merges ranked lists using only rank positions, which avoids having to calibrate a cosine similarity against a BM25 score. The question this project answers is whether RRF actually improves product search, and how the component rankers should be weighted. The main contributions are:

- A complete retrieval pipeline over the US portion of the Amazon ESCI dataset [5]: text preprocessing, three indexes over 482,105 products, RRF fusion, and a Streamlit search interface that shows the ESCI label next to each result.
- A full-catalog evaluation on all 8,956 test queries using Precision@10, Recall@100, MAP, nDCG@10 and 11-point precision-recall curves, under both strict (Exact only) and lenient (Exact or Substitute) relevance.
- A weighted RRF tuned only on training queries, together with a paired significance test against BM25.
- A query-level error analysis that shows where each ranker breaks down and why fusion helps.

Section 2 describes the dataset, Section 3 the pipeline and models, Section 4 the experiments and results, and Section 5 concludes with limitations and future work.

2. About Dataset

The Shopping Queries Dataset (ESCI) was released by Amazon for the KDD Cup 2022 [5]. It contains real customer queries in English, Spanish and Japanese, each paired with a set of candidate products that annotators judged with one of four labels: **Exact** (the product meets every requirement in the query), **Substitute** (it misses some requirement but could be used instead), **Complement** (it does not match, but would be used together with a matching product) and **Irrelevant**. We use the US locale of the "small" version, the harder query subset used for the ranking task, with its official train/test split. The searchable catalog consists of every US product judged for any of these queries, indexed on title, brand and color.

2.1 Dataset Statistics and Visualization

Table 1. Amazon ESCI, US locale (small version): dataset statistics.

Statistic	Train	Test	Total
Queries	20,888	8,956	29,844
Judged query-product pairs	419,653	181,701	601,354
Judgments per query (mean)	20.1	20.3	20.2
Words per query (mean / median)			3.9 / 4
Products in searchable catalog			482,105
Words per product text (mean)			19.2

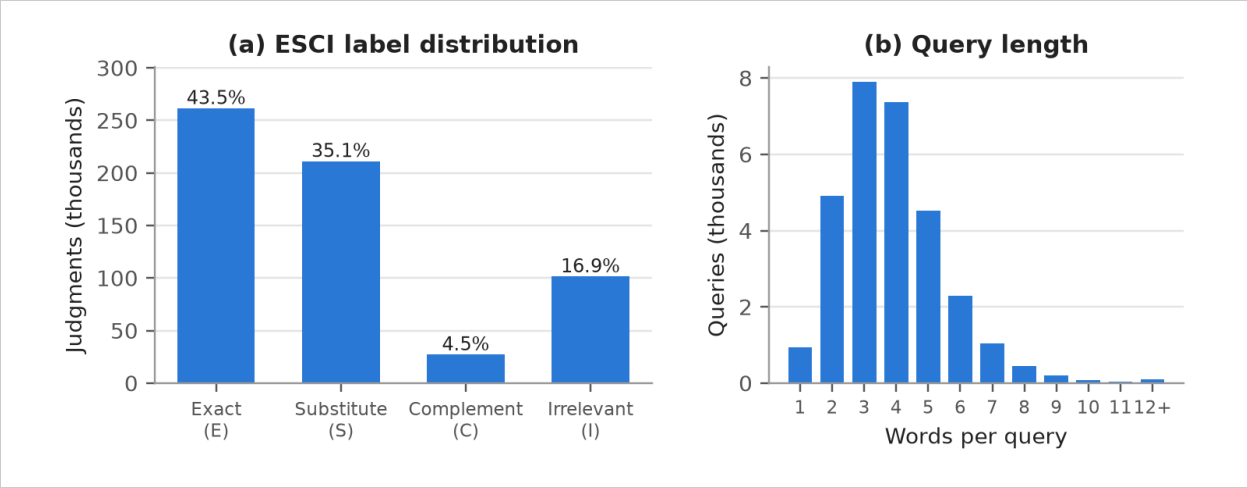


Figure 1: (a) ESCI label distribution over all 601,354 US judgments. (b) Query length in words over the 29,844 queries.

Figure 1(a) shows that only 43.5% of judged products are Exact matches, while 35.1% are Substitutes. A large share of the candidates a real search engine surfaced were therefore near misses, which is why we report results under both strict and lenient relevance. Complements are rare (4.5%). The train and test splits have almost the same label mix (Exact 43.3% versus 43.9%). Figure 1(b) confirms that queries are short: the median is four words and 94% of queries have six words or fewer. Product texts are also brief (19 words on average), and 32% of products have no color and 5.6% no brand. Each query has a median of 16 judged products (range 8 to 188), compared with a catalog of 482,105. Most products a system retrieves for a query are therefore unjudged; Section 3.5 explains how the evaluation treats them.

2.2 Example

Table 2 shows four of the judged products for the test query "razer headset without mic", one per label.

Table 2. Judged products for the test query "razer headset without mic" (titles shortened).

Label	Product title
Exact	Razer Kraken X Lite Ultralight Gaming Headset: 7.1 Surround Sound …
Substitute	Razer Kaira Pro Wireless Gaming Headset for Xbox Series X|S, Xbox One …
Complement	Razer Kitty Ears for Kraken Headsets: Compatible with Kraken 2019, Kraken TE …
Irrelevant	Razer Tetra Streaming Headset: Lightweight Frame, Bendable Cardioid Microphone …

All four titles contain both "Razer" and "headset", so counting shared words cannot separate them. The Irrelevant product is a headset whose title advertises a microphone, the one feature the shopper asked to avoid; the Complement is an accessory for a Razer headset; the Substitute is a different headset model. Getting this query right requires understanding negation and product type, which is exactly where the methods in this report struggle (Section 4.4).

3. Methodology

Every method shares the same catalog, preprocessing and evaluation code, so differences in the results come only from the ranking model. Figure 2 (end of Section 3.1) shows the full pipeline from an incoming query to the final ranking. The implementation is in Python 3.12 using scikit-learn, bm25s [6], sentence-transformers and Streamlit.

3.1 Preprocessing and Indexing

Each product is represented by its title, brand and color joined into one string. The lexical pipeline used by TF-IDF and BM25 unescapes HTML entities, removes tags, lowercases, replaces every non-alphanumeric character with a space, splits on whitespace, removes the 318 English stopwords in scikit-learn's list, and applies the Snowball (Porter2) stemmer. Queries go through the same steps, so "Headsets" and "headset" match. The dense model receives the cleaned text without stemming or stopword removal, since a transformer tokenizer expects natural text.

TF-IDF is stored as a sparse term-by-product matrix in compressed sparse row form, which works as an inverted index: each row lists the products containing that term. BM25 uses bm25s, which precomputes the score contribution of every (term, product) pair at indexing time, so a query only sums a few sparse rows. The dense index is a 482,105 × 384 matrix of L2-normalised float16 embeddings (about 370 MB). Building all three indexes is a one-off cost; encoding the catalog with the dense model takes about eight minutes on an Apple M1.

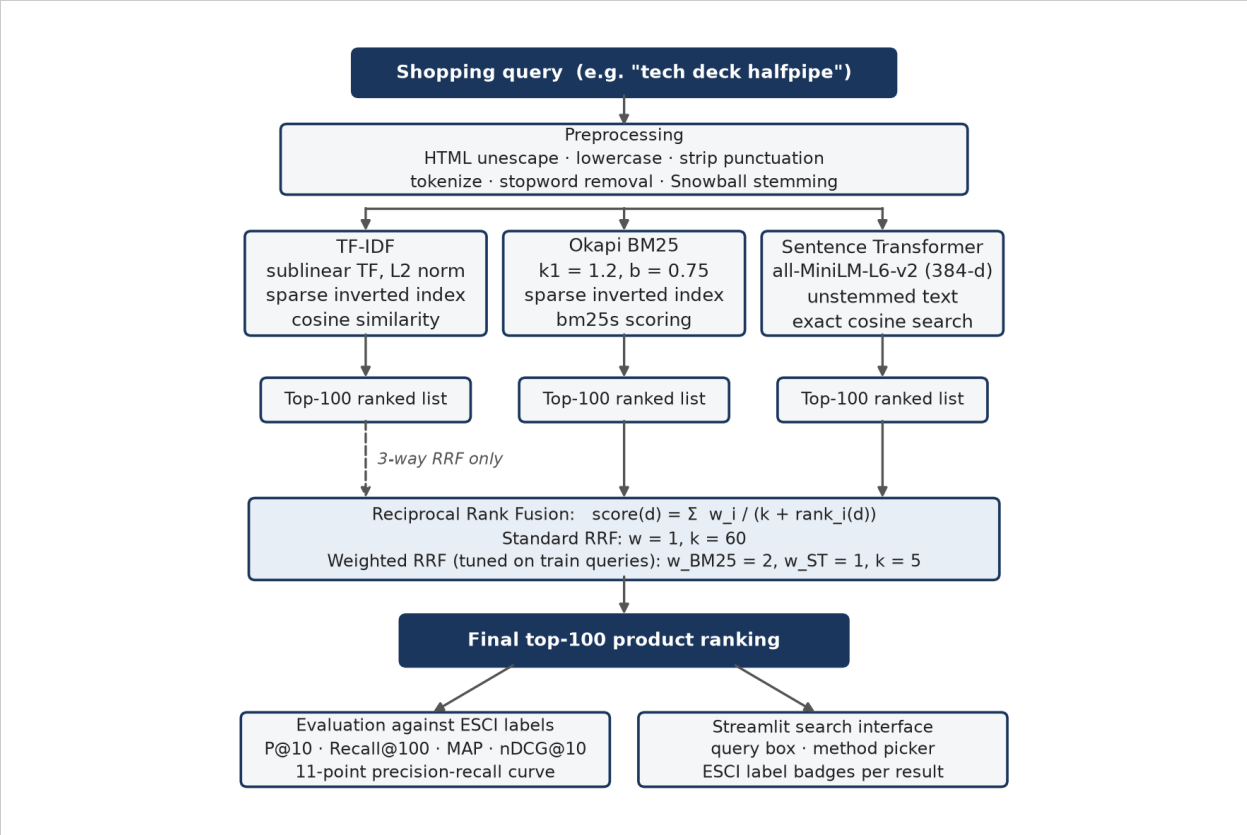


Figure 2: The retrieval pipeline. Three rankers each return their top 100 products, and Reciprocal Rank Fusion merges the lists.

3.2 Ranking Models

We compare three base rankers and three fusions of them:

- **TF-IDF Vector Space Model:** term weights use sublinear term frequency (1 + log tf) and smoothed inverse document frequency; vectors are L2-normalised and the score is the cosine between query and product.
- **Okapi BM25:** a probabilistic model with term-frequency saturation and length normalisation [1], using k_1 = 1.2, b = 0.75 and a Lucene-style IDF.
- **Sentence Transformer (dense):** all-MiniLM-L6-v2, a 6-layer MiniLM [7] bi-encoder that produces 384-dimensional embeddings and was trained on over a billion general-domain sentence pairs. It is used as-is, without fine-tuning on ESCI. Scores are cosine similarities from an exact (brute-force) search.
- **Hybrid RRF:** three fusions are evaluated: RRF (BM25 + ST) and RRF (TF-IDF + BM25 + ST) with standard settings, and a Weighted RRF (BM25 + ST) tuned as described in Section 3.4.

For BM25 the score of product d for query q is:

$$BM25(q,d) = \sum_{t \in q} IDF(t) \cdot \frac{tf(t,d) \cdot (k_1 + 1)}{tf(t,d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{avgdl})}$$

where tf(t, d) is the frequency of term t in d, |d| is the product length in terms and avgdl is the average length. Saturation means a title that repeats a word five times is not five times as relevant, which matters for keyword-stuffed product titles.

3.3 Reciprocal Rank Fusion Formulation

Each base ranker returns its top 100 products. RRF assigns every product a score based on its position in each list. For a set of rankers M, weights w_m and a constant k:

$$RRF(d) = \sum_{m \in M} \frac{w_m}{k + r_m(d)}$$

where $r_m(d)$ is the 1-based rank of d in ranker m's list; a product missing from a list contributes nothing for that ranker. Standard RRF uses $w_m = 1$ and k = 60 [4]. The constant k controls how much the top positions dominate: with k = 60 the first-ranked product scores only 15% more than the tenth (1/61 versus 1/70), whereas with k = 5 it scores 2.5 times more (1/6 versus 1/15). Because only ranks are used, the cosine similarities of the dense model and the unbounded BM25 scores never need to be put on a common scale.

3.4 Tuning the Weighted RRF

Equal weights assume both rankers are equally reliable, which Section 4 shows is not the case. We therefore searched over seven BM25 weights {1, 1.25, 1.5, 2, 2.5, 3, 4} (with the dense weight fixed at 1) and six values of k {5, 10, 20, 40, 60, 100}, 42 settings in all, on 2,000 randomly sampled **training** queries, selecting by nDCG@10. None of the models in this project is trained on ESCI, so the training split serves as a clean development set and the test split is never used for any decision.

The best setting is $w_{BM25} = 2$, $w_{ST} = 1$, k = 5 (development nDCG@10 = 0.3387, against 0.3275 for standard RRF). Figure 3 (below) shows a broad plateau: 19 of the 42 settings are within 0.002 of the best, spanning BM25 weights from 1.25 to 4 and k from 5 to 60, so the gain does not depend on a lucky choice. Equal weights are the worst row. The clear pattern is that BM25 should count more than the dense model, and that the top of each list should count more than standard RRF allows.

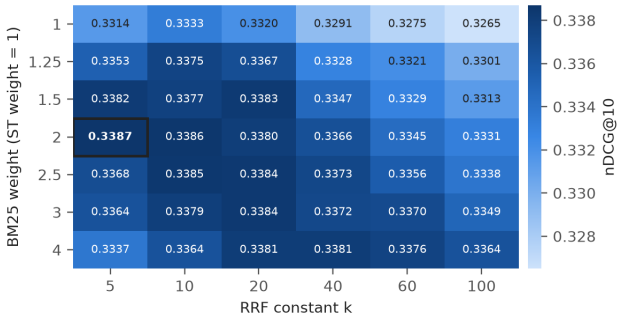


Figure 3: nDCG@10 of weighted RRF (BM25 + ST) on 2,000 training queries for each BM25 weight and k. The selected setting is outlined.

3.5 Evaluation Protocol and Metrics

Each method retrieves the top 100 products for each test query **from the full catalog** of 482,105 products, not just from that query's judged candidates. Binary metrics are computed under two definitions of relevance: **strict** (only Exact is relevant) and **lenient** (Exact or Substitute). We report Precision@10, Recall@100, Mean Average Precision over the top 100 (MAP) and an 11-point interpolated precision-recall curve. nDCG@10 [8] uses the graded gains from the KDD Cup: Exact = 1.0, Substitute = 0.1, Complement = 0.01, Irrelevant = 0. Retrieved products with no judgment for the query count as non-relevant. Since only about 20 of 482,105 products are judged per query, absolute scores are a lower bound on true quality, but every method is measured the same way, so comparisons between methods remain fair. Statistical significance is tested with a paired t-test and a 95% bootstrap confidence interval (2,000 resamples) over per-query differences [9].

3.6 Search Interface

The Streamlit app lets a user type a query or pick any labelled test query, choose a ranking method (weighted RRF by default) and the number of results. Each result shows the product title (linked to Amazon), brand, color and product ID. For test queries the app also shows the ESCI label of each result as a colored badge (or "unjudged"), together with P@10 and nDCG@10 for that query. A second tab displays the full comparison table and precision-recall curves.

4. Experimental Setup and Performance Analysis

All experiments run on an Apple M1 laptop with 8 GB of memory. Queries are processed in batches and latency is reported per query; a hybrid's latency includes running each of its component rankers. The strict evaluation covers the 8,955 test queries that have at least one Exact product, and the lenient evaluation all 8,956.

4.1 Results

Table 3. Strict relevance (Exact = relevant), 8,955 test queries, full-catalog retrieval.

Ranking model	P@10	Recall@100	MAP	nDCG@10	ms / query
TF-IDF	0.1538	0.5096	0.1802	0.2301	1.7
BM25	0.2071	0.5669	0.2503	0.3196	0.5
Sentence Transformer	0.1710	0.5205	0.1988	0.2633	5.7
RRF (BM25 + ST)	0.2079	0.6041	0.2498	0.3195	6.2
RRF (TF-IDF + BM25 + ST)	0.2037	0.5890	0.2423	0.3112	7.9
Weighted RRF (BM25 + ST)	**0.2155**	**0.6013**	**0.2605**	**0.3313**	**6.2**

Table 4. Lenient relevance (Exact or Substitute = relevant), 8,956 test queries.

Ranking model	P@10	Recall@100	MAP	nDCG@10
TF-IDF	0.2203	0.4291	0.1700	0.2301
BM25	0.2972	0.4851	0.2364	0.3196
Sentence Transformer	0.2521	0.4508	0.1943	0.2633
RRF (BM25 + ST)	0.3020	0.5207	0.2427	0.3195
RRF (TF-IDF + BM25 + ST)	0.2950	0.5048	0.2331	0.3112
Weighted RRF (BM25 + ST)	**0.3096**	**0.5175**	**0.2490**	**0.3313**

nDCG@10 uses graded gains in both tables, so it is identical across Tables 3 and 4. The highlighted row is the best overall method; it is best on every metric except Recall@100, where standard RRF is slightly higher.

BM25 is the strongest single ranker and also the fastest. Compared with TF-IDF it improves P@10 by 35% and nDCG@10 by 39% (Table 3). Both see exactly the same tokens, so the difference comes from how they weight them, mainly BM25's term-frequency saturation and its length normalisation, which is gentler on longer titles than TF-IDF's cosine normalisation.

The Sentence Transformer beats TF-IDF but trails BM25, which scores 21% higher in nDCG@10. all-MiniLM-L6-v2 was trained on general sentence pairs, not shopping data, and shopping queries rely heavily on brand names, model numbers and sizes, where an exact token match is the strongest evidence available. It is also the slowest base ranker, at 5.7 ms per query for an exact search over 482K vectors.

Standard RRF adds recall but not precision. Fusing BM25 with the dense model raises strict Recall@100 from 0.567 to 0.604 (+6.6%), because the dense model finds relevant products that BM25 never retrieves. P@10, MAP and nDCG@10, however, stay level with BM25. With equal weights and k = 60, a product ranked first by the weaker dense model is worth almost as much as one ranked first by BM25, which offsets the benefit at the top of the list.

Weighted RRF is the best method overall. Doubling BM25's weight and using k = 5 keeps nearly all of the recall gain (0.601 versus 0.567 for BM25) and also improves the top of the ranking: nDCG@10 +3.7%, MAP +4.1% and P@10 +4.1% relative to BM25. Against the dense model alone, it gains 26% in nDCG@10. The same ordering holds under lenient relevance (Table 4).

Adding TF-IDF to the fusion hurts. The three-way RRF is worse than the two-way one on every metric (nDCG@10 0.311 versus 0.320). TF-IDF is a noisier version of the same lexical signal as BM25, so including it counts that signal twice and pulls the fused ranking towards the weaker lexical model. Fusion helps when the inputs make different mistakes, not when more rankers are added.

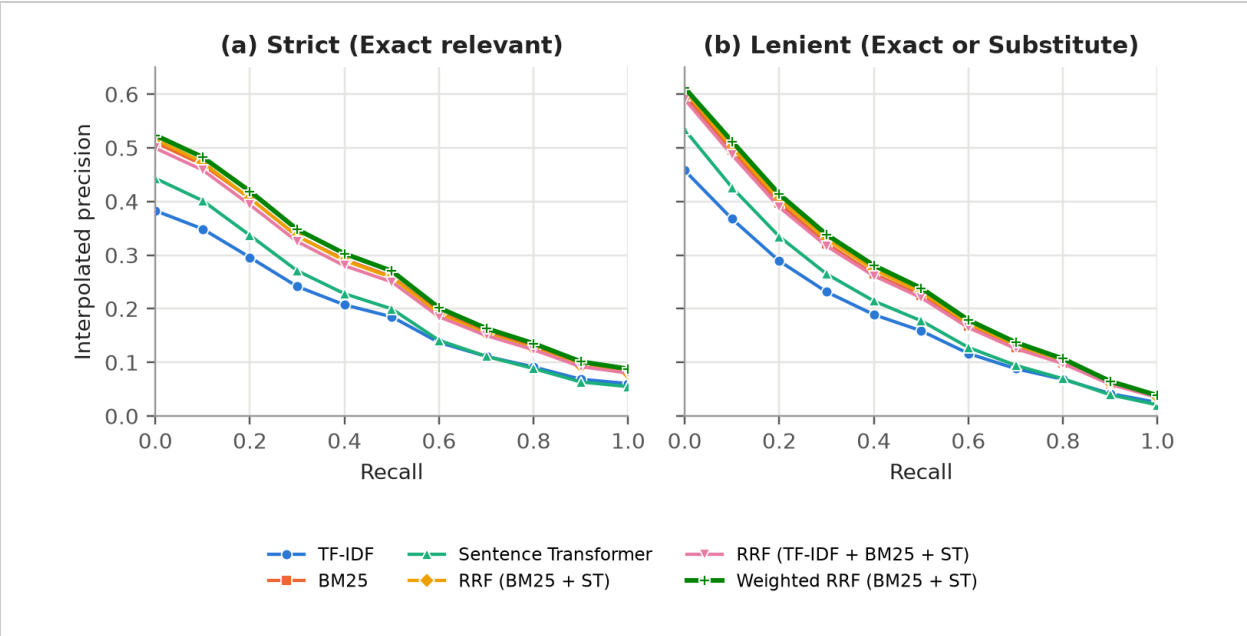


Figure 4: 11-point interpolated precision-recall curves for all six methods (top 100, full-catalog retrieval). BM25 and the two-way RRF curves lie almost on top of each other.

The precision-recall curves in Figure 4 tell the same story. Weighted RRF lies on or above every other curve at all recall levels under both relevance definitions. BM25 and the hybrids form a tight group well above the dense model and TF-IDF at low recall, which corresponds to the first few results a shopper actually sees. At high recall the dense model and TF-IDF converge, since both run out of relevant products within the top 100.

4.2 Statistical Significance

A difference of 0.012 in nDCG@10 is small in absolute terms, so we tested whether it is reliable. Over the 8,955 strict test queries, weighted RRF improves per-query nDCG@10 over BM25 by +0.0117 on average (95% bootstrap CI [+0.0103, +0.0131], paired t-test $p = 8.1 \times 10^{-56}$). It wins on 3,393 queries, loses on 2,668 and ties on 2,894. For MAP the gain is +0.0102 (95% CI [+0.0089, +0.0115], $p = 7.0 \times 10^{-54}$), with 4,165 wins and 3,162 losses. The improvement is therefore consistent across thousands of queries rather than driven by a few outliers, although the many losses show that the right ranker still varies from query to query.

4.3 Efficiency

BM25 answers a query in 0.5 ms, more than ten times faster than the dense model. A hybrid costs roughly the sum of its parts (6.2 ms for BM25 + ST), since fusing two lists of 100 is negligible. For a production system the dense search would use an approximate nearest-neighbour index, but even exact search keeps every method well under 10 ms per query on a laptop.

4.4 Error Analysis

Table 5. Per-query nDCG@10 for selected test queries.

Query	TF-IDF	BM25	ST	RRF	W-RRF
sanus blt2-b1	0.930	0.918	0.025	0.416	0.636
hp 201a black	0.578	0.847	0.000	0.114	0.812
bill nyhe	0.000	0.000	0.469	0.469	0.469
your mine and ours	0.000	0.000	0.595	0.595	0.595
razer headset without mic	0.057	0.164	0.011	0.050	0.059

Table 5 shows typical failure cases. On model-number queries such as "sanus blt2-b1" and "hp 201a black", the lexical rankers are near perfect while the dense model finds nothing relevant, because to a general-purpose encoder one printer cartridge code looks much like another. Standard RRF gives the dense model's wrong products as much credit as BM25's right ones and drops to 0.114 on the cartridge query; weighted RRF keeps most of BM25's quality (0.812). This is the mechanism behind the overall gain.

The reverse happens when preprocessing removes the useful words. scikit-learn's stopword list contains "bill", so "bill nyhe" (a misspelling of Bill Nye) is reduced to the single misspelled token "nyhe". The book title "your mine and ours" consists entirely of stopwords and becomes an empty query (preprocessing empties two other test queries as well). The dense model, which sees the original text, still finds relevant products, and RRF passes its result through because BM25 returns nothing to compete with it.

Finally, "razer headset without mic" defeats every method. The lexical pipeline removes "without" as a stopword (as it does "not" and "no"); 723 test queries (8%) contain one of these three words. The dense model keeps the word but, like most bi-encoders, does not reliably model negation, and ranks headsets with microphones first. Fusion cannot recover a signal that none of its inputs captures.

5. Conclusion

This project built and evaluated a product search system on the Amazon ESCI dataset, comparing TF-IDF, BM25, a Sentence Transformer and three Reciprocal Rank Fusion hybrids on 8,956 real shopping queries over a catalog of 482,105 products. Three findings stand out. First, BM25 remains a very strong baseline for product search: it is the best and fastest single ranker, well ahead of an off-the-shelf dense model, because shopping queries depend on exact tokens such as brands and model numbers. Second, fusion helps only when it is weighted sensibly. Standard RRF improves recall by 6.6% but not the top of the ranking, whereas a weighted RRF that trusts BM25 twice as much and emphasises top ranks ($k = 5$) improves nDCG@10, MAP and P@10 by about 4% over BM25. The gain is statistically significant, and the tuning surface is flat enough that the setting is not fragile. Third, adding more rankers is not automatically better: including TF-IDF alongside BM25 lowered every metric.

The work has limitations. ESCI judges only about 20 products per query, so relevant products retrieved from the rest of the catalog are counted as wrong, and the absolute scores understate true quality. The study covers only the US English subset and indexes only title, brand and color, leaving out product descriptions and bullet points. The dense model was not adapted to the shopping domain, and the general-purpose stopword list removes words such as "without", "not" and "bill" that carry meaning in product queries.

These limitations suggest clear next steps. Fine-tuning the bi-encoder on the ESCI training pairs, using the graded labels as targets, should close much of the gap to BM25 and make fusion more valuable. A cross-encoder re-ranker applied to the fused top 100 could model negation and the Exact-versus-Substitute distinction directly. A shorter, product-aware stopword list that keeps negations is a cheap fix for the lexical rankers. Finally, the fusion weights could depend on the query, for example trusting BM25 more for queries that contain model numbers and the dense model more for descriptive queries.

References

1. Robertson, S., Zaragoza, H.: The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 3(4), 333–389 (2009)
2. Salton, G., Buckley, C.: Term-weighting approaches in automatic text retrieval. Information Processing & Management 24(5), 513–523 (1988)
3. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: Proceedings of EMNLP-IJCNLP, pp. 3982–3992 (2019)
4. Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. In: Proceedings of SIGIR, pp. 758–759 (2009)
5. Reddy, C.K., Màrquez, L., Valero, F., Rao, N., Zaragoza, H., Bandyopadhyay, S., Biswas, A., Xing, A., Subbian, K.: Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search. arXiv:2206.06588 (2022)
6. Lù, X.H.: BM25S: Orders of magnitude faster lexical search via eager sparse scoring. arXiv:2407.03618 (2024)
7. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M.: MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In: Advances in NeurIPS 33 (2020)
8. Järvelin, K., Kekäläinen, J.: Cumulated gain-based evaluation of IR techniques. ACM Transactions on Information Systems 20(4), 422–446 (2002)
9. Smucker, M.D., Allan, J., Carterette, B.: A comparison of statistical significance tests for information retrieval evaluation. In: Proceedings of CIKM, pp. 623–632 (2007)